An End-to-End AutoML Pipeline for Time-Series and Tabular Data

Architecture, Optimization, and Application to Financial Forecasting

Lenny Zerbib Maxence Tarlet Mohamed Abdellahi Berenice Maire

Machine Learning for Finance — 2026

Agenda

1. **Motivation**
2. **System Architecture**
3. **Optimization with Optuna**
4. **Cross-Validation Design**
5. **Experiment System**
6. **Web Interface**
7. **QRT Challenge Application**
8. **Results & Conclusion**

From raw CSV to trained model in a single pipeline — no manual preprocessing, no manual tuning.

1. Motivation

Why build an AutoML pipeline?

The Problem with Manual ML Workflows

What practitioners do today

- Choose a model family manually
- Tune hyperparameters by trial and error
- Decide preprocessing steps case by case
- Re-implement CV logic for each project
- Track experiments in spreadsheets

What our pipeline does

- User uploads a CSV and sets a few options
- Automatic preprocessing (numeric, cat., text)
- Joint model selection + hyperparameter search
- Time-series-aware or standard CV built in
- Auto-logging to `results.csv`

Result

Slow, hard to reproduce, prone to data leakage

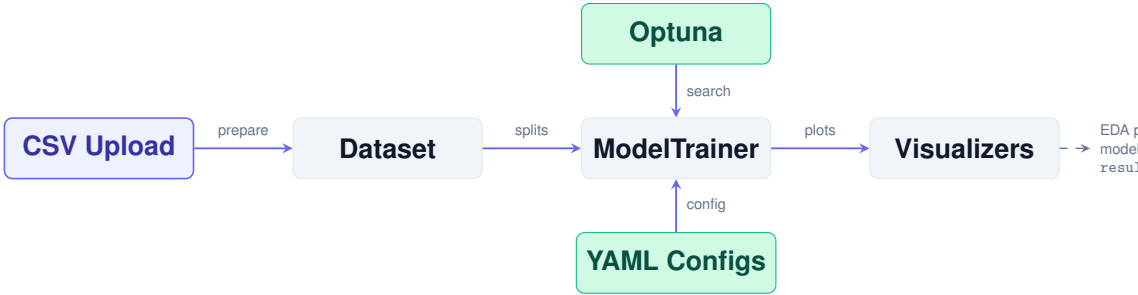
Goal

Reproducible, end-to-end framework from ingestion to evaluation

2. System Architecture

Four modules, one pipeline

Architecture Overview



preparing.py
Ingestion, split, preprocessing

modeling.py
Optuna, fitting, logging

visualizing.py
EDA, confusion matrix, importances

main.py
FastAPI, dashboard

Dataset Module — `preparing.py`

Core responsibilities

- Chronological sort + datetime parsing
- Target encoding (LabelEncoder for strings)
- Train/test split — shuffle OFF for time series
- ColumnTransformer: numeric, categorical, text

Preprocessing choices

- Numeric: zero impute (panel-safe) or KNN
- Categorical: mode impute + OneHotEncoder

Two entry points

`Dataset(df, target, ...)`

Standard: full DataFrame, splits internally

`Dataset.from_split(X_train, ...)`

Competition: pre-split, fit on train only

Feature engineering helpers

- `add_lag_features()` — shift within group
- `add_rolling_features()` — shift before window

3. Optimization

Joint model selection and hyperparameter search

Optimization Strategy — Optuna

Objective function

- Classification: maximize mean AUC across folds
- Regression: minimize mean RMSE across folds
- Early stopping on LGB and XGB within each fold

Pruning strategy

- `MedianPruner`: drops underperforming trials
- 5-trial warm-up before pruning activates
- Intermediate scores reported per fold

Model families searched jointly

Boosting **LightGBM**

n_estimators, lr, num_leaves, max_depth, reg

Boosting **XGBoost**

n_estimators, lr, max_depth, reg_alpha/lambda

Ensemble **Random Forest**

n_estimators, max_depth, min_samples

Linear **Logistic Reg. + Ridge**

C, penalty {L1/L2}, alpha

4. Cross-Validation

Preventing leakage

Cross-Validation Design

Standard CV (tabular data)

- StratifiedKFold (clf) or KFold (reg)
- $k = \min(5,\ \text{min class size})$
- Shuffle optional, user-controlled

Inside Optuna

- Each trial = full k-fold on training set
- Best params $\Rightarrow$ refit on entire train
- Test set touched only at final evaluation

Date-based CV (time series)

- All rows from same date stay in same fold
- Prevents look-ahead bias in panel data
- Model cloned and refit on each local fold

5. Experiment System

YAML configs for manual control

YAML Experiment System — Three Modes

Mode 1 — Free search

No YAML. Optuna explores all families freely.

Use when: exploring a new dataset.

Mode 2 — Fixed params

`params` key in YAML. Skips Optuna entirely.

Use when: reproducing a known-good config.

Mode 3 — Optuna hints

`optuna_hints` key. Seeds first trial, then searches freely.

Use when: exploring around a good baseline.

Example — fixed params (Mode 2)

```
is_classification: true
model: lgb
params:
  n_estimators: 477
  learning_rate: 0.067
  num_leaves: 122
```

Auto-logging to `results.csv`

- `experiment_id` (timestamp)
- `config` (YAML filename)
- `model` (class name)
- `accuracy` / `auc` / `rmse` / `r2`

6. Web Interface

The interactive dashboard

FastAPI Dashboard

Two endpoints

- `POST /upload` — accepts CSV, returns columns, dtypes, 5-row preview
- `POST /run` — full pipeline, returns JSON with logs + base64 plots

User controls

- Target column, timestamp, feature subset
- Toggles: Time Series, Classification, Shuffle, Scale, KNN
- Sliders: Test size, Validation size, Optuna trials

Plot capture mechanism

```
def _fake_show(*args, **kwargs):
  fig = plt.gcf()
  if fig.get_axes():
    figures.append(
      _fig_to_b64(fig))
    plt.close(fig)
plt.show = _fake_show
```

Captures all matplotlib figures as base64 PNG, bundled in the JSON response and rendered in the browser.

7. QRT Challenge Application

Financial forecasting — beating the benchmark

QRT Asset Allocation Challenge

Problem setup

- Panel: 511 dates $\times$ 278 allocations $\approx$ $105{,}000$ rows
- Each row = a trading strategy at a date
- Target: sign of next-day return

$$\hat{y} = \mathbf{1}\left[r_{S,t+1} > 0\right]$$

- Metric: directional accuracy
- Benchmark (LightGBM): **50.79%**

Key features

- `RET_1` to `RET_20` — lagged returns
- `SIGNED_VOLUME_1` to `_20` — directional flow
- `MEDIAN_DAILY_TURNOVER`
- `GROUP` — anonymized strategy family
- `TS` — anonymized, shuffled dates

Key constraint

Dates are **shuffled and anonymized**. No temporal ordering guaranteed. Standard k-fold is as valid as a time split.

EDA — Key Findings

Missing Values

`SIGNED_VOLUME_1` missing in **72.8%** of rows.

Date-level: all allocations on affected dates lack it simultaneously.

$\Rightarrow$ Excluded. Imputation unreliable.

Momentum Signal

Strong momentum at lag 1:

Quintile	Win rate
Q5 (top)	**54.06%**
Q1 (bot)	48.47%

Mild mean-reversion at lags 2 and 4.

Turnover + Volatility

High turnover: **52.19%** win rate vs 50.28%.

High recent volatility: **52.61%** vs 50.05%.

$\Rightarrow$ Both carry signal, included as features.

Feature Engineering — Five Families

Family	Features	Rationale
Momentum	`MOM_3`, `MOM_5`, `MOM_vs_MEAN`, `SHARPE_5`	Short-term trend strength
Volatility	`VOL_5`, `VOL_20`, `VOL_RATIO`	Regime detection
Volume	`SVOL_SHORT`, `CONVICTION` = RET1 × SVOL2	Directional flow and conviction
Cross-sect. (date)	`CS_RANK_RET1`, `CS_RANK_VOL5`	Relative positioning at same timestamp
Cross-sect. (group)	`CS_GROUP_MEAN_RET1`, `CS_GROUP_RANK_RET1`, ...	Intra-group relative standing

Cross-sectional ranks are the most robust features: they adjust automatically for market-wide regime shifts.

Results — Model Comparison

Configuration	CV Acc.	Test Acc.	AUC
LightGBM benchmark (provided)	N/A	50.79%	N/A
Logistic Regression (L2)	50.41%	50.35%	0.503
Random Forest (200 trees)	51.12%	50.88%	0.511
XGBoost (Optuna, 50 trials)	51.87%	51.43%	0.517
LightGBM (fixed best params)	52.10%	51.89%	0.523
LightGBM (Optuna, 30 trials)	**52.40%**	**52.15%**	**0.528**

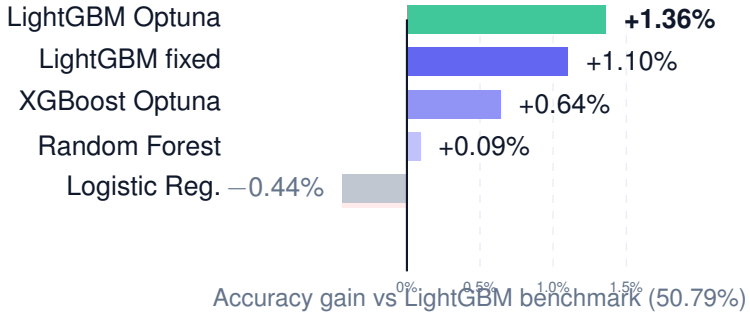
Key drivers

- Cross-sectional rank features (top importance)
- `SHARPE_5` and `MOM_vs_MEAN`
- Optuna over fixed params: $+0.26$pp

Why +1.36pp matters

- Near-50% win rate: every fraction counts
- At scale, 1–2% accuracy $\Rightarrow$ direct P&L
- Consistent gain across CV folds

Accuracy Gain vs Benchmark



8. Conclusion

What we built, what we learned

Conclusion

What we built

- End-to-end AutoML: ingestion → evaluation
- Joint model + hyperparameter search (Optuna)
- Date-based CV — time-series-safe
- YAML experiment system (3 modes) + auto-logging
- FastAPI dashboard for non-technical users

QRT result

- **52.40% CV**, **52.15% test accuracy**
- Benchmark 50.79% ⇒ **+1.36pp**

Limitations

- No Optuna warm-starting across sessions
- YAML not exposed in the web UI
- No multi-output target support

Future work

- Optuna study persistence (SQLite)
- YAML mode in the dashboard
- Walk-forward CV for ordered datasets
- Factor model features for QRT

Thank you

Questions?

Lenny Zerbib Maxence Tarlet Mohamed Abdellahi Berenice Maire